

LLM 推理基础

目 录

1 引言：为什么推理和训练不同	2
2 自回归生成：逐 token 的概率链	2
2.1 直觉	2
2.2 形式化	2
3 Prefill 与 Decode：两阶段推理	3
3.1 Prefill 阶段	3
3.2 Decode 阶段	3
4 KV Cache：用空间换时间	3
4.1 动机	3
4.2 原理	3
4.3 KV Cache 大小	4
5 性能指标：衡量推理速度的标尺	4
6 实战：小模型算一算	5
7 要点速查	5
8 prefill 延迟	5
9 单步 decode 延迟	5
10 本章你将学会	6
11 小结	6

1 引言：为什么推理和训练不同

训练大语言模型时，我们一次性把整个数据集灌进 GPU，前向算 loss、反向算梯度，整个过程像流水线运转，每个样本都经过相同的计算图。但推理时情况完全不同。

我们不知道用户会输入什么 prompt，也不知道模型会生成多长的回答。更关键的是，模型必须一个字一个字地“吐”出来，每生成一个 token，都要把之前所有的 token 重新“看”一遍。这种逐 token 生成的模式叫做**自回归生成**（autoregressive generation），它是 LLM 推理区别于训练的根本原因。

自回归带来两个核心挑战：

1. **计算重复**：每生成一个新 token，都要重新计算之前所有 token 的注意力。不做缓存的话，序列长度 S 的生成复杂度是 $O(S^2)$ 。
2. **并行度差异**：处理 prompt 时可以一次算很多 token，但生成时每次只算一个，GPU 利用率天差地别。

这两个挑战催生了本章的核心技术：**KV Cache**（键值缓存），以及两阶段推理流程：**Prefill**（预填充）和 **Decode**（解码）。理解这些概念是后续优化 Gemma4 推理性能的基础。

2 自回归生成：逐 token 的概率链

2.1 直觉

不妨把 LLM 想象成接龙游戏：你给出一句话的开头，模型根据已有内容预测下一个字，把预测的字拼回去，再预测下一个，如此往复。每一步的输出都依赖之前所有步的输出，形成一条链条。

2.2 形式化

给定 token 序列 $x_1, x_2, \dots, x_t$ ，模型预测下一个 token x_{t+1} 的概率分布：

$$p(x_{t+1} \mid x_1, x_2, \dots, x_t) \quad (2.1)$$

这就是自回归生成的数学本质。模型不是一次性生成整句话，而是逐步采样：在第 t 步，用前 t 个 token 作为输入，计算第 $t+1$ 个 token 的分布，采样后拼入序列，再进入第 $t+1$ 步。

直觉 每一步的输出都依赖之前所有步的输出，构成一条马尔可夫链。我们不能跳过中间步骤直接生成第 t 步的输出，因为第 t 步的输入包含了第 $t-1$ 步的输出。

从模型内部看，每一步生成都经过完整的 Transformer 前向传播：

1. **Embedding**：把 token id 映射为向量。
2. **多层 Attention + FFN**：提取上下文表示。
3. **LM Head**：把最后一层隐状态映射为词表大小的 logits。
4. **Sampling**：从 logits 采样出下一个 token id。

关键瓶颈在 Attention 计算。标准自注意力的公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.2)$$

其中 Q, K, V 分别是 Query、Key、Value 矩阵， d_k 是每个头的维度。如果每次生成都从头算所有 token 的 K 和 V ，计算量随序列长度平方增长。这就是 KV Cache 要解决的问题。

3 Prefill 与 Decode：两阶段推理

现代 LLM 推理框架把生成过程分成两个阶段，每个阶段有不同的计算特征。

3.1 Prefill 阶段

Prefill 处理用户输入的整个 prompt。在这个阶段，模型一次性计算所有 prompt token 的表示，把每层每个 token 的 Key 和 Value 写入 cache。

直觉 Prefill 就像考试前通读一遍题目：你需要把每道题都看一遍，建立整体理解。这一步 token 多，但可以高度并行，GPU 跑起来很爽。

Prefill 的计算特征：

- **矩阵乘规模大**：输入是 $S \times D$ 的矩阵，相当于一次大矩阵乘法。
- **GPU 并行度高**：多个 token 的注意力可以并行计算。
- **计算密集**：每读取一个权重，要做多次乘加运算。

Prefill 的延迟就是 **TTFT** (Time To First Token, 首 token 延迟)，决定用户等待第一个字出现的时间。

3.2 Decode 阶段

Decode 逐个生成 token。每一步只输入上一个生成的 token，用 KV Cache 中的历史信息计算注意力。

直觉 Decode 就像答题过程：题目已经读过了 (prefill)，现在每写一个字只需要“瞄一眼”之前的笔记 (KV Cache)，然后写出下一个字。

Decode 的计算特征：

- **矩阵乘退化为矩阵-向量乘**：输入只有 $1 \times D$ ，权重矩阵 $D \times D$ 不变，但每次只算一个向量。
- **GPU 利用率低**：大量权重需要从显存读取，但只做很少的计算，瓶颈在访存。
- **访存密集**：每生成一个 token，都要把整个模型的权重读一遍。

Decode 的单步延迟就是 **TPOT** (Time Per Output Token, 每 token 延迟)，决定生成速度。用户感受到的生成速度 (tokens/s) 约等于 $\frac{1}{\text{TPOT}}$ 。

4 KV Cache：用空间换时间

4.1 动机

自回归生成中，已经处理过的 token 的 Key 和 Value 向量不会改变。每步都重新计算它们是巨大浪费。KV Cache 的思路很简单：把每层每个 token 的 K 和 V 存起来，下一步直接使用。

4.2 原理

Transformer 有 N 层，每层有 H_{kv} 个 KV 头 (GQA 下可能比 Query 头少)，每个头的维度是 D_h 。对于序列中的每个 token，我们存储它的 Key 和 Value。

- **Prefill 时**：把整个 prompt 的所有 KV 一次性写入 cache。
- **Decode 时**：每生成一个 token，只写入该 token 的 1 组 KV。

4.3 KV Cache 大小

KV Cache 的显存占用公式为：

$$M_{kv} = \frac{2BSNH_{kv}D_h \cdot y}{8} \text{ bytes} \quad (4.1)$$

每个符号的含义：

符号	含义	说明
B	batch size	同时处理的请求数
S	sequence length	最大序列长度
N	层数	Transformer 的层数
H_{kv}	KV 头数	GQA 下可能少于 H_q
D_h	每头维度	head dimension
y	激活位宽	BF16 为 16, INT8 为 8
2	Key 和 Value	两组向量

公式中的 2 不是 batch size 的 B ，而是因为每个 token 既有 Key 又有 Value，要存两份。完整计数：2 (K 和 V) $\times B \times S \times N \times H_{kv} \times D_h$ 个元素，每个元素 $\frac{y}{8}$ 字节。

5 性能指标：衡量推理速度的标尺

推理性能有四个核心指标，每个对应不同的用户体验维度。

指标	全称	含义
TTFT	Time To First Token	prefill 延迟
TPOT	Time Per Output Token	单步 decode 延迟
Token 吞吐量	tokens/s	每秒生成的 token 数
请求吞吐量	requests/s	每秒完成的请求数

TTFT 影响首屏延迟感受，TPOT 影响阅读流畅度。在 lab5 的推理框架中，RequestMetrics 记录了这些指标：

```
@dataclass
class RequestMetrics:
    ttft_s: float
    mean_tpot_s: float
    prefill_latency_s: float
    decode_latencies_s: list[float]
```

ttft_s 是首 token 延迟，mean_tpot_s 是平均每 token 延迟，prefill_latency_s 是 prefill 阶段总延迟，decode_latencies_s 记录每步 decode 的延迟列表。框架输出的性能摘要中，generated_tokens_per_s 反映 token 吞吐量，requests_per_s 反映请求吞吐量。

6 实战：小模型算一算

我们用一个玩具模型来感受 prefill 和 decode 的差异。假设模型有 2 层，隐藏维度 $D = 64$ ，4 个注意力头 ($H_q = H_{kv} = 4$)，每头维度 $D_h = 16$ ，词表大小 $V = 1000$ ，权重和激活都是 BF16 ($y = 16$)。用户输入 8 个 token 的 prompt。

例 KV Cache 大小：

$$M_{kv} = \frac{2 \times 1 \times 8 \times 2 \times 4 \times 16 \times 16}{8} = 4096 \text{ bytes} = 4 \text{ KiB} \quad (6.1)$$

即存储 8 个 token 的 KV Cache 只需 4 KiB。

Prefill 一步的注意力 MACs (2 层合计)：

$$QK^T: S \times S \times D_h \times H \times N = 8 \times 8 \times 16 \times 4 \times 2 = 8192$$

AV: 同样 8192

合计 16384 MACs。

Decode 一步的注意力 MACs (2 层合计)：

$$QK^T: 1 \times S \times D_h \times H \times N = 1 \times 8 \times 16 \times 4 \times 2 = 1024$$

AV: 同样 1024

合计 2048 MACs。

比值： $\frac{16384}{2048} = 8 = S$ 。Prefill 做了 S 倍的计算，但两者读取的权重完全相同。这意味着 prefill 的算术强度 (FLOPS/byte) 是 decode 的 S 倍，prefill 是计算密集型，decode 是访存密集型。

7 要点速查

概念	定义	公式/说明
自回归生成	逐 token 预测	$p(x_{t+1} x_1, \dots, x_t)$
Prefill	处理 prompt	计算密集, 决定 TTFT
Decode	逐 token 生成	访存密集, 决定 TPOT
KV Cache	缓存 K 和 V	避免重复计算
TTFT	首 token 延迟	8 prefill 延迟
TPOT	每 token 延迟	9 单步 decode 延迟
Token 吞吐量	tokens/s	$\approx \frac{1}{\text{TPOT}}$
请求吞吐量	requests/s	每秒完成请求数

Prefill 与 Decode 的关键对比：

特征	Prefill	Decode
输入 token 数	S (多个)	1 (单个)
矩阵乘类型	矩阵 $\times$ 矩阵	矩阵 $\times$ 向量
GPU 利用率	高	低
瓶颈	计算	访存

特征	Prefill	Decode
算术强度	$O(S)$	$O(1)$

10 本章你将学会

1. 解释 LLM 推理与训练的本质区别，说出自回归生成的定义和挑战。
2. 描述 Prefill 和 Decode 两阶段的计算特征，区分 TTFT 和 TPOT。
3. 推导 KV Cache 大小公式，计算给定模型配置下的 KV Cache 显存占用。
4. 使用 RequestMetrics 的字段解释推理框架输出的性能指标。

11 小结

我们从“为什么推理和训练不同”出发，理解了自回归生成的本质：每一步都依赖之前所有步的输出。这种依赖催生了两个阶段，Prefill 一次处理整个 prompt，计算密集；Decode 逐 token 生成，访存密集。KV Cache 用空间换时间，把已计算的 Key 和 Value 存起来避免重复计算，代价是显存随序列长度线性增长。

下一章我们将深入 Gemma4-12B 的模型架构，看看滑动窗口注意力、GQA 和 SwiGLU 如何影响 KV Cache 的大小和推理效率。

讲义基于 HPC Lab5 实验指导编写